

Threads (III): Una race condition

Lezione 16

Sistemi Operativi - II Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Un esempio «insidioso» di race condition con flussi di esecuzione (threads) concorrenti



Flussi di esecuzione concorrente

- La fonte di maggior complicazione nei sistemi operativi è costituita dall'esistenza di flussi di esecuzione concorrenti:
 - In una **applicazione multithread**, ciascun thread è un flusso di esecuzione “in competizione” con quelli degli altri thread
 - In un sistema multiprogrammato con prelazione, ciascun processo può essere interrotto da un altro processo
 - In un sistema multiprocessore, diversi processi o gestori di interruzione sono in esecuzione contemporaneamente
- La coerenza delle **strutture di dati private** di ciascun flusso di esecuzione è garantita dal meccanismo di cambio di flusso (contesto)
- La coerenza delle **strutture di dati condivise** tra i vari flussi di esecuzione concorrenti **non è garantita** da tale meccanismo

Esempio

Flusso di `main()`

```
...  
for (i = 0; i < T; i++) {  
⇒ ...  
⇒ s=pthread_create(&tid[i], NULL, job, &i);  
  ...  
}  
...
```

Memoria di `main()`

0

i

Funzione per nuovi flussi

```
void *job(void *arg) {  
    int *p=arg;  
    printf("%d", *p);  
}
```

Esempio

Flusso di `main()`

```
...  
for (i = 0; i < T; i++) {  
    ...  
    s=pthread_create(&tid[i], NULL, job, &i);  
    ...  
}  
...
```

Funzione per nuovi flussi



```
void *job(void *arg) {  
    int *p=arg;  
    printf("%d", *p);  
}
```

Memoria di `main()`

Memoria di `job()`

0

i



arg



p

Esempio

Flusso di `main()`

```
...  
for (i = 0; i < T; i++) {  
    ...  
    s=pthread_create(&tid[i], NULL, job, &i);  
    ...  
}  
...
```

Memoria di `main()`

0

i

Funzione per nuovi flussi

```
void *job(void *arg) {  
    → int *p=arg;  
    printf("%d", *p);  
}
```

Memoria di `job()`



arg



p

Esempio

Flusso di `main()`

```
...  
for (i = 0; i < T; i++) {  
    ...  
    s=pthread_create(&tid[i], NULL, job, &i);  
    ...  
}  
...
```

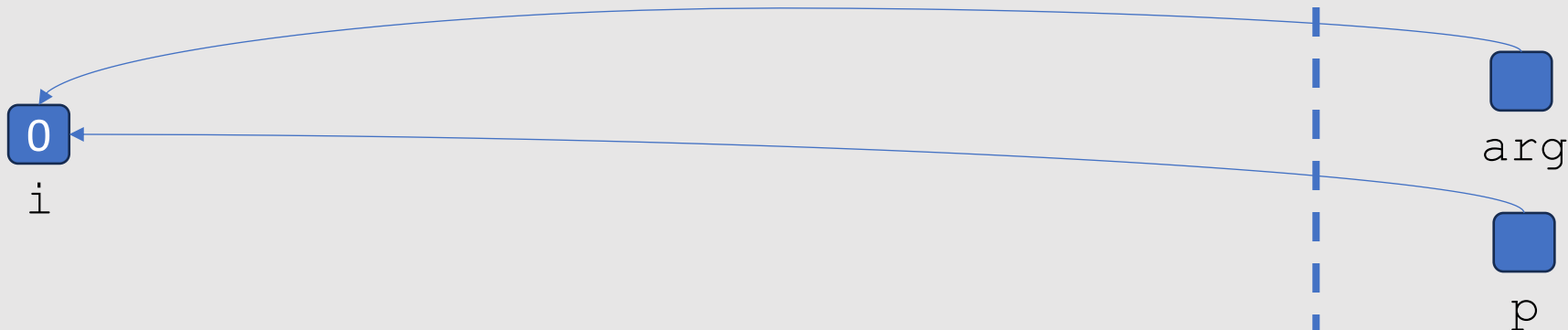


Funzione per nuovi flussi

```
void *job(void *arg) {  
    int *p=arg;  
    printf("%d", *p);  
}
```

Memoria di `main()`

Memoria di `job()`



Esempio

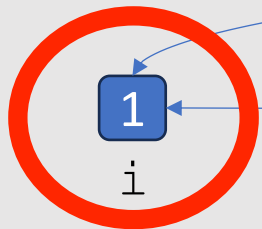
Flusso di `main()`

```
...  
→ for (i = 0; i < T; i++) {  
→   ...  
   s=pthread_create(&tid[i], NULL, job, &i);  
   ...  
}  
...
```

Funzione per nuovi flussi

```
void *job(void *arg) {  
    int *p=arg;  
    printf("%d", *p);  
}
```

Memoria di `main()`



Memoria di `job()`

A diagram showing a memory location for variable `arg` in the `job()` function. It consists of a blue square with the label `arg` written below it. A blue arrow points from this variable to the `i` variable in the `main()` function's memory.

A diagram showing a memory location for variable `p` in the `job()` function. It consists of a blue square with the label `p` written below it. A blue arrow points from this variable to the `i` variable in the `main()` function's memory.

Esempio

Flusso di `main()`

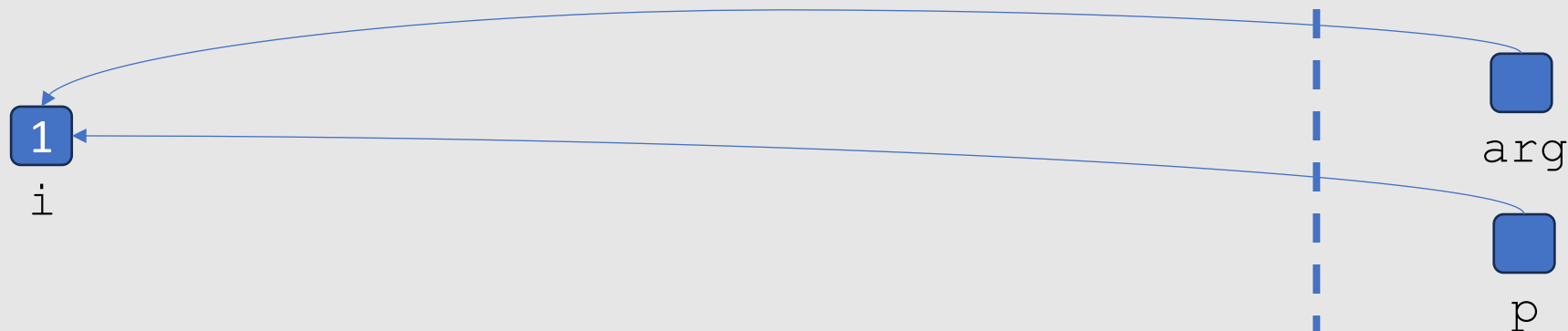
```
...  
for (i = 0; i < T; i++) {  
    ...  
    s=pthread_create(&tid[i], NULL, job, &i);  
    ...  
}  
...
```

Funzione per nuovi flussi

```
void *job(void *arg) {  
    int *p=arg;  
    → printf("%d", *p);  
}
```

Memoria di `main()`

Memoria di `job()`



Esempio

Flusso di `main()`

```
...  
for (i = 0; i < T; i++) {  
    ...  
    s=pthread_create(&tid[i], NULL, job, &i);  
    ...  
}  
...
```

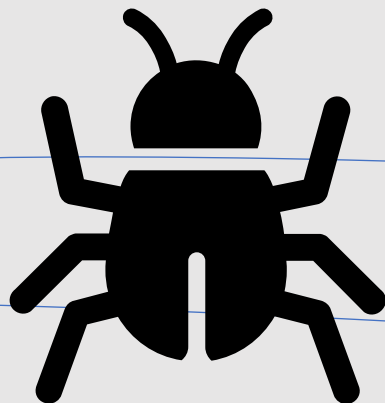
Funzione per nuovi flussi

```
void *job(void *arg) {  
    int *p=arg;  
    → printf("%d", *p);  
}
```

Memoria di `main()`

1

i



Memoria di `job()`



arg



p

Il flusso di `job()` «vede» 1 invece di 0!

Cosa fare?

- Evitare soluzioni «complicate»
- Preferire soluzioni concettualmente semplici (ed eventualmente inefficienti) rispetto a soluzioni «efficienti» ma di cui non si è certi della correttezza
- Nel nostro caso, meglio inizializzare la memoria condivisa nella parte seriale del codice

Domande?

